SAGA PATTERN

in Spring Boot Microservices

The Complete Visual Guide

Layman → Technical → HLD → LLD → Code

Includes: Why separate DBs? | Kafka late-read problem solved | Compensating transactions

Table of Contents

Chapter 1 — The Simple Story — Layman Language
What is this, and WHY does it exist?

Chapter 2 — The Problem it Solves
Single DB vs Multiple DBs explained

Chapter 3 — High Level Design (HLD)
How many services, how they connect

Chapter 4 — Saga Flow — Step by Step
What happens on success and failure

Chapter 5 — The Kafka Late-Read Problem & Outbox Pattern
The biggest real-world challenge

Chapter 6 — Low Level Design (LLD)
Database schemas, APIs, topics

Chapter 7 — Complete Java Code — Order Service
Every line explained

Chapter 8 — Complete Java Code — Inventory Service
Every line explained

Chapter 9 — Complete Java Code — Payment Service
Every line explained

Chapter 10 — Idempotency — Handling Duplicate Events
What it is and why it matters

Chapter 11 — Summary & When to Use / Avoid
Quick reference card

Chapter 1

The Simple Story — Layman Language

IMAGINE THIS REAL LIFE SITUATION

You go to a restaurant. You order food, the waiter writes it down (Order taken). The kitchen starts cooking (Stock reserved). You pay the bill (Payment done). Then the waiter confirms your order is complete.

Now imagine: you paid, but the kitchen ran out of ingredients AFTER you already paid. What should happen? The restaurant should REFUND your money and CANCEL the order. That undo-process is called a **Compensating Transaction** in the Saga pattern.

In software: when you buy something on Amazon, there are 3 separate computer systems — one for Orders, one for Warehouse (Inventory), one for Payments. They do NOT share one giant database. Each has its own. The Saga Pattern coordinates them.

IN ONE LINE

Saga Pattern = Do things step by step. If any step fails, automatically UNDO all previous steps.

WHY CAN'T WE USE ONE DATABASE FOR EVERYTHING?

This is the most common confusion. The answer is simple:

- Each microservice is a SEPARATE application, possibly on a different server, even in a different city.
- If all services share one DB, they become tightly coupled — fixing one breaks another.
- Microservices are meant to be INDEPENDENT. Independent deployment, independent scaling.
- One DB = one point of failure. Your entire app dies if that DB has issues.
- Amazon's Order team and Payment team are different departments — they own their own data.

SHARED DATABASE (Wrong)	SEPARATE DATABASE (Saga Way)
All services read/write same DB	Each service owns its DB only
Change one table = might break others	Change is isolated to that service
One DB crash = entire app down	One service down = others still work
Hard to scale — you scale all or nothing	Scale Payment service separately from Order
NOT microservices — this is a distributed monolith	True microservices architecture

Chapter 2

The Problem Saga Solves

BEFORE SAGA — THE PROBLEM (without any pattern)

Imagine you wrote this naive code (the WRONG way):

```
// BAD CODE - Without Saga Pattern
@PostMapping("/order")
public String createOrder(OrderRequest req) {
    // Step 1: Save order
    orderService.save(req);                    // Saved in Order DB ✓

    // Step 2: Reserve inventory
    inventoryService.reserve(req.productId);   // Reserved in Inventory DB ✓

    // Step 3: Charge payment -- CRASH HAPPENS HERE!
    paymentService.charge(req.amount);         // SERVER CRASH or TIMEOUT ✗

    // Now: Order is saved, Inventory is reserved, but Payment NEVER happened.
    // Customer is confused. Money not taken. But stock is locked. DATABASE IS INCONSISTENT!
    return "Order placed";
}
```

■ WARNING: This is called DATA INCONSISTENCY. Your databases are now in an illegal state. This is the exact problem the Saga Pattern was invented to solve.

AFTER SAGA — THE SOLUTION

- Each step is a separate local transaction in its own database.
- Each step publishes an event to Kafka when it succeeds.
- The next step only starts when it receives that event.
- If any step FAILS, it publishes a FAILURE event.
- The previous steps then run COMPENSATING transactions to undo themselves.

■ TIP: Think of it like a relay race. Each runner (service) runs their part and passes the baton (event). If a runner drops the baton, the race is STOPPED and everyone returns to start position.

Chapter 3

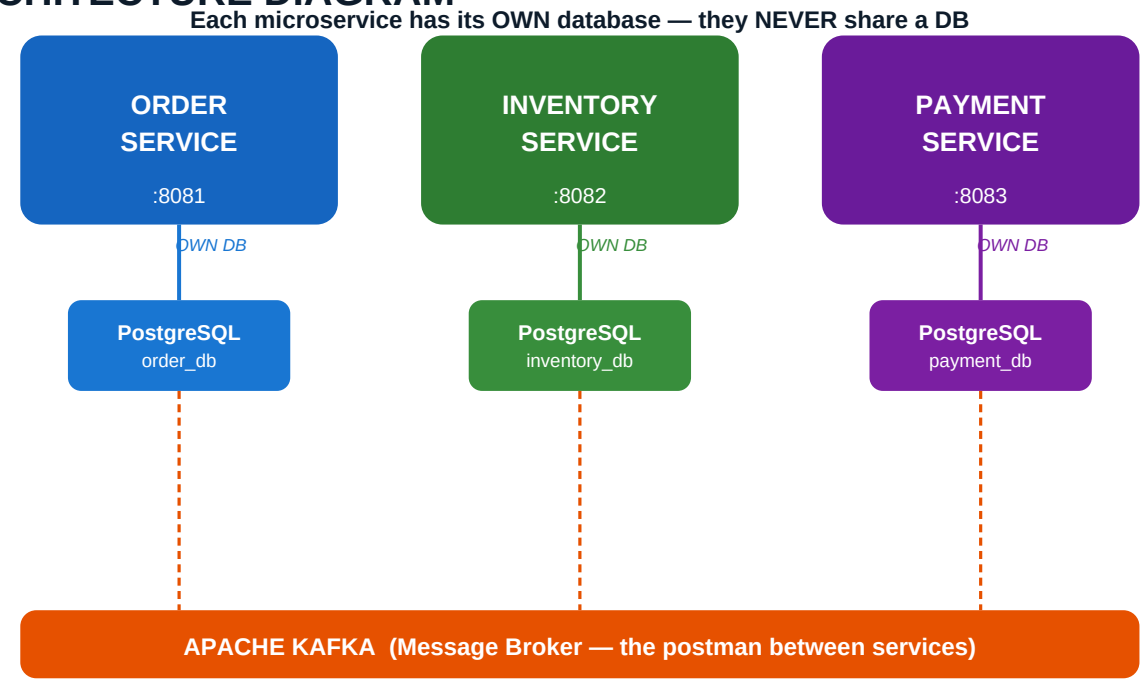
High Level Design (HLD)

HOW MANY MICROSERVICES AND HOW THEY CONNECT

Our e-commerce order system has exactly 4 microservices and 1 message broker:

Service	Port	Responsibility	Database
API Gateway	:8080	Single entry point for all client requests. Routes to correct service.	No DB
Order Service	:8081	Creates orders. Orchestrates the entire saga. Tracks saga state.	order_db (PostgreSQL)
Inventory Service	:8082	Reserves and releases product stock.	inventory_db (PostgreSQL)
Payment Service	:8083	Charges customer. Issues refunds when needed.	payment_db (PostgreSQL)

ARCHITECTURE DIAGRAM



KAFKA TOPICS (the communication channels)

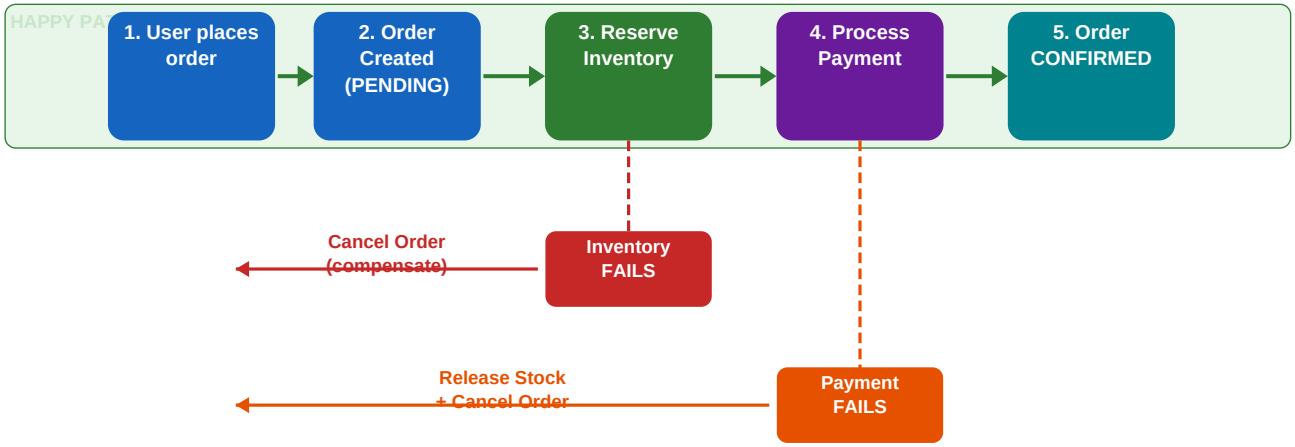
Kafka Topic	Published By	Consumed By	Meaning
order-created	Order Service	Inventory Service	Please reserve stock for this order
inventory-reserved	Inventory Service	Order Service	Stock reserved OK, proceed to payment
inventory-failed	Inventory Service	Order Service	Stock unavailable, cancel the saga
payment-requested	Order Service	Payment Service	Charge the customer now
payment-completed	Payment Service	Order Service	Payment OK, confirm the order

payment-failed	Payment Service	Order Service	Payment failed, release stock + cancel order
inventory-release	Order Service	Inventory Service	UNDO: Release the reserved stock (compensation)

Chapter 4

Saga Flow — Step by Step

COMPLETE SAGA FLOW DIAGRAM



Dashed lines = Failure path Solid arrows = Success path Arrows going left = Compensation (UNDO)

STEP BY STEP — HAPPY PATH (Everything succeeds)

Step	Action	Who Does It	What Happens in DB
Step 1	User calls POST /orders via API Gateway	Order Service	Order saved with status=PENDING in order_db
Step 2	Order Service publishes 'order-created' event to Kafka	Kafka	Event contains: orderId, productId, quantity, amount
Step 3	Inventory Service consumes 'order-created'	Inventory Service	Checks stock. If available, reduces quantity in inventory_db
Step 4	Inventory Service publishes 'inventory-reserved' event	Kafka	Signals Order Service that stock is locked
Step 5	Order Service consumes 'inventory-reserved'	Order Service	Publishes 'payment-requested' to Kafka
Step 6	Payment Service consumes 'payment-requested'	Payment Service	Calls bank API, saves transaction in payment_db
Step 7	Payment Service publishes 'payment-completed'	Kafka	Signals Order Service that money is taken

Step 8	Order Service consumes 'payment-completed'	Order Service	Updates order status to CONFIRMED in order_db. SAGA COMPLETE!

COMPENSATION PATH (When Payment Fails)

Step	Action	Who Does It	What Happens in DB
Step 1-4	Same as above	—	Order PENDING, stock reserved
Step 5	Payment Service charges bank — FAILS	Payment Service	Saves FAILED status in payment_db
Step 6	Payment Service publishes 'payment-failed'	Kafka	Contains reason for failure
Step 7	Order Service consumes 'payment-failed'	Order Service	Publishes 'inventory-release' event
Step 8	Inventory Service consumes 'inventory-release'	Inventory Service	Adds stock back in inventory_db (UNDO step 3)
Step 9	Order Service updates order	Order Service	Status = CANCELLED in order_db. SAGA COMPENSATED!

Chapter 5

Kafka Late-Read Problem & Outbox Pattern

THE PROBLEM: WHAT IF KAFKA IS SLOW OR DOWN?

This is the BIGGEST real-world challenge with Saga + Kafka. Here is the scenario:

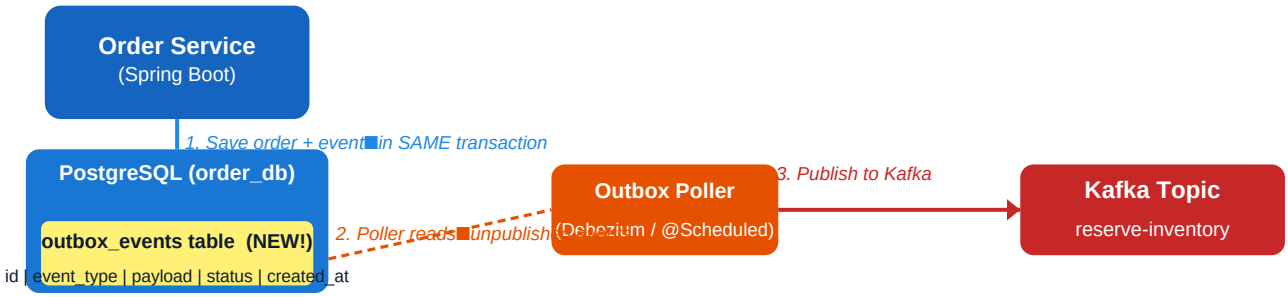
```
// ORDER SERVICE does this:
orderRepository.save(order);           // 1. Saves order to DB  ✓
kafkaTemplate.send("order-created");  // 2. Sends to Kafka     ✗ <-- Kafka is temporarily DOWN

// What happens?
// - Order IS saved in the database
// - Event is NOT in Kafka
// - Inventory Service never receives the event
// - Saga is STUCK. Order stays PENDING forever.
// - Manual intervention needed. This is a BUG.
```

■ WARNING: This is called the DUAL WRITE PROBLEM. You are writing to two separate systems (DB and Kafka) and they are NOT in the same transaction. One can succeed while the other fails.

THE SOLUTION: OUTBOX PATTERN

Instead of writing to DB and Kafka separately, you write to DB ONLY. A separate poller (Debezium or @Scheduled) then reads from the DB and publishes to Kafka. Both the business data and the event go into the SAME database transaction.



Result: Even if Kafka is DOWN at step 3, the event is safe in DB and will be published when Kafka recovers. Zero data loss.

HOW THE OUTBOX TABLE LOOKS

```
-- outbox_events table (in order_db, same database as orders table)
CREATE TABLE outbox_events (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_id BIGINT NOT NULL,        -- orderId
    event_type   VARCHAR(100) NOT NULL,  -- e.g. 'ORDER_CREATED'
    payload      JSONB NOT NULL,         -- full event data as JSON
    status       VARCHAR(20) DEFAULT 'PENDING',  -- PENDING or PUBLISHED
    created_at   TIMESTAMP DEFAULT NOW(),
    published_at TIMESTAMP
```

```
);
```

HOW THE ORDER SERVICE SAVES DATA (Same Transaction)

```
@Service
@Transactional   // <-- ONE SINGLE TRANSACTION covers BOTH saves below
public class OrderService {

    @Autowired private OrderRepository orderRepo;
    @Autowired private OutboxRepository outboxRepo;

    public Order createOrder(CreateOrderRequest req) {
        // Save the actual order
        Order order = new Order(req.getProductId(), req.getQuantity(),
                                req.getAmount(), OrderStatus.PENDING);
        order = orderRepo.save(order);  // Saved to orders table

        // Save the event in the SAME transaction (not to Kafka yet!)
        OutboxEvent event = new OutboxEvent(
            order.getId(),
            "ORDER_CREATED",
            toJson(new OrderCreatedPayload(order.getId(),
                                           req.getProductId(),
                                           req.getQuantity()))
        );
        outboxRepo.save(event);  // Saved to outbox_events table

        // Both saves are in ONE @Transactional block.
        // Either BOTH succeed or BOTH fail. No inconsistency possible.
        return order;
    }
}
```

THE OUTBOX POLLER (reads DB and publishes to Kafka)

```
@Component
public class OutboxEventPoller {

    @Autowired private OutboxRepository outboxRepo;
    @Autowired private KafkaTemplate<String, String> kafkaTemplate;

    @Scheduled(fixedDelay = 1000)  // runs every 1 second
    @Transactional
    public void pollAndPublish() {
        // Find all events not yet sent to Kafka
        List<OutboxEvent> pending = outboxRepo
            .findByStatusOrderByCreatedAtAsc("PENDING");

        for (OutboxEvent event : pending) {
            try {
                // Determine topic from event type
                String topic = resolveTopicName(event.getEventType());

                // Publish to Kafka
                kafkaTemplate.send(topic, event.getPayload()).get();
                // .get() makes it synchronous - waits for Kafka acknowledgment

                // Mark as published
                event.setStatus("PUBLISHED");
                event.setPublishedAt(LocalDateTime.now());
                outboxRepo.save(event);

            } catch (Exception e) {
```

```
                // Kafka still down? Event stays PENDING, will retry next poll
                log.warn("Failed to publish event {}, will retry", event.getId());
            }
        }
    }

    private String resolveTopicName(String eventType) {
        return switch (eventType) {
            case "ORDER_CREATED"       -> "order-created";
            case "PAYMENT_REQUESTED"   -> "payment-requested";
            case "INVENTORY_RELEASE"   -> "inventory-release";
            default -> throw new IllegalArgumentException("Unknown: " + eventType);
        };
    }
}
```

■ TIP: With the Outbox Pattern, Kafka can be DOWN for hours. Events accumulate in the DB and are automatically published when Kafka recovers. Zero message loss. Zero manual fix needed.

Chapter 6

Low Level Design (LLD)

DATABASE SCHEMAS

Order Service — order_db

```
-- orders table
CREATE TABLE orders (
    id            BIGSERIAL PRIMARY KEY,
    product_id    VARCHAR(50) NOT NULL,
    quantity      INT NOT NULL CHECK (quantity > 0),
    total_price   DECIMAL(10,2) NOT NULL,
    status        VARCHAR(20) NOT NULL,   -- PENDING/CONFIRMED/CANCELLED
    created_at    TIMESTAMP DEFAULT NOW(),
    updated_at    TIMESTAMP DEFAULT NOW()
);

-- saga_state table (tracks where in the saga we are)
CREATE TABLE saga_state (
    order_id      BIGINT PRIMARY KEY REFERENCES orders(id),
    current_step  VARCHAR(50),            -- ORDER_CREATED, INVENTORY_RESERVED, etc.
    saga_status   VARCHAR(20),            -- IN_PROGRESS, COMPLETED, COMPENSATING, FAILED
    failure_reason VARCHAR(255),
    updated_at    TIMESTAMP DEFAULT NOW()
);

-- outbox_events table (already shown in Chapter 5)
```

Inventory Service — inventory_db

```
CREATE TABLE products (
    id                  BIGSERIAL PRIMARY KEY,
    product_id          VARCHAR(50) UNIQUE NOT NULL,
    name                VARCHAR(200) NOT NULL,
    available_quantity  INT NOT NULL CHECK (available_quantity >= 0),
    reserved_quantity   INT NOT NULL DEFAULT 0
);

-- reservation table (important for compensation!)
CREATE TABLE reservations (
    id          BIGSERIAL PRIMARY KEY,
    order_id    BIGINT UNIQUE NOT NULL,   -- UNIQUE prevents double-reservation
    product_id  VARCHAR(50) NOT NULL,
    quantity    INT NOT NULL,
    status      VARCHAR(20) NOT NULL,     -- RESERVED / RELEASED
    created_at  TIMESTAMP DEFAULT NOW()
);
```

Payment Service — payment_db

```
CREATE TABLE payments (
    id              BIGSERIAL PRIMARY KEY,
    order_id        BIGINT UNIQUE NOT NULL,  -- UNIQUE prevents double-charge!
    amount          DECIMAL(10,2) NOT NULL,
    status          VARCHAR(20) NOT NULL,    -- PENDING/COMPLETED/FAILED/REFUNDED
    gateway_ref     VARCHAR(100),            -- external payment ID
```

```
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);
```

REST APIS EXPOSED BY EACH SERVICE

Service	Method	Endpoint	Description
Order	POST	/api/orders	Create new order (starts saga)
Order	GET	/api/orders/{id}	Get order with current saga status
Order	GET	/api/orders/{id}/saga	Get full saga execution history
Inventory	GET	/api/inventory/{productId}	Check available stock
Payment	GET	/api/payments/{orderId}	Get payment status for order

Chapter 7

Order Service — Complete Code

FILE STRUCTURE OF ORDER SERVICE

```
order-service/
 src/main/java/com/saga/order/
  ■■■ OrderServiceApplication.java      <- Spring Boot main class
  ■■■ controller/
  ■   ■■■ OrderController.java          <- REST endpoints (POST /orders)
  ■■■ service/
  ■   ■■■ OrderService.java             <- Business logic + start saga
  ■   ■■■ SagaOrchestrator.java         <- Kafka listeners + saga management
  ■■■ entity/
  ■   ■■■ Order.java                    <- JPA entity for orders table
  ■   ■■■ SagaState.java                <- JPA entity for saga_state table
  ■   ■■■ OutboxEvent.java              <- JPA entity for outbox_events table
  ■■■ repository/
  ■   ■■■ OrderRepository.java
  ■   ■■■ SagaStateRepository.java
  ■   ■■■ OutboxRepository.java
  ■■■ events/                           <- Event classes (POJOs)
  ■   ■■■ OrderCreatedEvent.java
  ■   ■■■ InventoryReservedEvent.java
  ■   ■■■ InventoryFailedEvent.java
  ■   ■■■ PaymentCompletedEvent.java
  ■   ■■■ PaymentFailedEvent.java
  ■■■ outbox/
      ■■■ OutboxEventPoller.java        <- @Scheduled Kafka publisher
```

ORDER ENTITY (Order.java)

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_id", nullable = false)
    private String productId;

    @Column(nullable = false)
    private int quantity;

    @Column(name = "total_price", nullable = false)
    private double totalPrice;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private OrderStatus status;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Always use constructor or builder - never expose setters for status
```

```
    // Status should only change through saga orchestrator
    @PrePersist
    protected void onCreate() { this.createdAt = LocalDateTime.now(); }

    // Getters and setters...
}

public enum OrderStatus { PENDING, CONFIRMED, CANCELLED }
```

SAGA ORCHESTRATOR (The Brain of the Saga)

This class LISTENS to Kafka events and decides what to do next. It is the central coordinator — think of it as the manager who tells each department what to do.

```
@Service
@Slf4j
public class SagaOrchestrator {

    @Autowired private KafkaTemplate<String, String> kafkaTemplate;
    @Autowired private OrderRepository orderRepo;
    @Autowired private SagaStateRepository sagaStateRepo;
    @Autowired private OutboxRepository outboxRepo;
    @Autowired private ObjectMapper objectMapper;

    // ------------------------------------------------------
    // LISTENER 1: Inventory confirmed stock is reserved
    // This is triggered AFTER Inventory Service reserves stock
    // ------------------------------------------------------
    @KafkaListener(topics = "inventory-reserved", groupId = "order-service-group")
    @Transactional
    public void onInventoryReserved(String message) {
        try {
            InventoryReservedEvent event = objectMapper.readValue(message,
                                                                  InventoryReservedEvent.class);
            Long orderId = event.getOrderId();
            log.info("Inventory reserved for order: {}", orderId);

            // Update saga state
            updateSagaState(orderId, "INVENTORY_RESERVED", "IN_PROGRESS");

            // Next step: request payment via Outbox (not directly to Kafka!)
            Order order = orderRepo.findById(orderId)
                .orElseThrow(() -> new RuntimeException("Order not found: " + orderId));

            saveOutboxEvent(orderId, "PAYMENT_REQUESTED",
                Map.of("orderId", orderId, "amount", order.getTotalPrice()));

        } catch (Exception e) {
            log.error("Error processing inventory-reserved: {}", e.getMessage());
        }
    }

    // ------------------------------------------------------
    // LISTENER 2: Inventory says stock is NOT available
    // ------------------------------------------------------
    @KafkaListener(topics = "inventory-failed", groupId = "order-service-group")
    @Transactional
    public void onInventoryFailed(String message) {
        InventoryFailedEvent event = parseEvent(message, InventoryFailedEvent.class);
        log.warn("Inventory failed for order: {} reason: {}",
                  event.getOrderId(), event.getReason());

        // No compensation needed (nothing was charged yet)
```

```
        // Just cancel the order
        cancelOrder(event.getOrderId(), event.getReason());
    }

    // ---------------------------------------------------------
    // LISTENER 3: Payment successful
    // ---------------------------------------------------------
    @KafkaListener(topics = "payment-completed", groupId = "order-service-group")
    @Transactional
    public void onPaymentCompleted(String message) {
        PaymentCompletedEvent event = parseEvent(message, PaymentCompletedEvent.class);
        log.info("Payment completed for order: {}", event.getOrderId());

        // SAGA SUCCESS! Confirm the order.
        orderRepo.findById(event.getOrderId()).ifPresent(order -> {
            order.setStatus(OrderStatus.CONFIRMED);
            orderRepo.save(order);
        });
        updateSagaState(event.getOrderId(), "PAYMENT_COMPLETED", "COMPLETED");
    }

    // ---------------------------------------------------------
    // LISTENER 4: Payment failed — need to COMPENSATE
    // Compensation = undo step 3 (release the reserved stock)
    // ---------------------------------------------------------
    @KafkaListener(topics = "payment-failed", groupId = "order-service-group")
    @Transactional
    public void onPaymentFailed(String message) {
        PaymentFailedEvent event = parseEvent(message, PaymentFailedEvent.class);
        log.warn("Payment failed for order: {}. Starting compensation...",
                 event.getOrderId());

        // Update saga to COMPENSATING state
        updateSagaState(event.getOrderId(), "PAYMENT_FAILED", "COMPENSATING");

        // COMPENSATING TRANSACTION: Tell Inventory Service to release stock
        Order order = orderRepo.findById(event.getOrderId()).get();
        saveOutboxEvent(event.getOrderId(), "INVENTORY_RELEASE",
            Map.of("orderId", event.getOrderId(),
                   "productId", order.getProductId(),
                   "quantity", order.getQuantity()));

        // Cancel the order
        cancelOrder(event.getOrderId(), "Payment failed: " + event.getReason());
    }

    // ---------------------------------------------------------
    // HELPER: Cancel order and mark saga as failed
    // ---------------------------------------------------------
    private void cancelOrder(Long orderId, String reason) {
        orderRepo.findById(orderId).ifPresent(order -> {
            order.setStatus(OrderStatus.CANCELLED);
            orderRepo.save(order);
        });
        updateSagaState(orderId, "ORDER_CANCELLED", "FAILED");
    }

    // ---------------------------------------------------------
    // HELPER: Save to outbox (not directly to Kafka!)
    // ---------------------------------------------------------
    private void saveOutboxEvent(Long orderId, String eventType, Map<String, Object> payload) {
        try {
```

```
            OutboxEvent outboxEvent = new OutboxEvent();
            outboxEvent.setAggregateId(orderId);
            outboxEvent.setEventType(eventType);
            outboxEvent.setPayload(objectMapper.writeValueAsString(payload));
            outboxEvent.setStatus("PENDING");
            outboxRepo.save(outboxEvent);
        } catch (Exception e) {
            throw new RuntimeException("Failed to save outbox event", e);
        }
    }
}
```

Chapter 8

Inventory Service — Complete Code

INVENTORY SERVICE LISTENS TO 2 TOPICS:

- 'order-created' — Reserve stock for the order
- 'inventory-release' — COMPENSATION: Release the reserved stock back

```
@Service
@Slf4j
public class InventoryService {

    @Autowired private ProductRepository productRepo;
    @Autowired private ReservationRepository reservationRepo;
    @Autowired private KafkaTemplate<String, String> kafkaTemplate;
    @Autowired private ObjectMapper objectMapper;

    // ------------------------------------------------------
    // FORWARD STEP: Reserve stock when order is created
    // ------------------------------------------------------
    @KafkaListener(topics = "order-created", groupId = "inventory-service-group")
    @Transactional
    public void onOrderCreated(String message) {
        OrderCreatedEvent event = parseEvent(message, OrderCreatedEvent.class);
        Long orderId = event.getOrderId();

        // IDEMPOTENCY CHECK: Have we already processed this order?
        // This prevents double-processing if Kafka delivers the same message twice
        if (reservationRepo.existsByOrderId(orderId)) {
            log.warn("Already processed reservation for order: {}. Skipping.", orderId);
            return;  // <-- This is idempotency in action
        }

        Product product = productRepo.findByProductId(event.getProductId())
            .orElse(null);

        if (product == null || product.getAvailableQuantity() < event.getQuantity()) {
            // NOT ENOUGH STOCK — publish failure event
            publishEvent("inventory-failed",
                Map.of("orderId", orderId,
                       "reason", product == null ? "Product not found" : "Insufficient stock"));
            return;
        }

        // Reserve the stock (reduce available, increase reserved)
        product.setAvailableQuantity(
            product.getAvailableQuantity() - event.getQuantity());
        product.setReservedQuantity(
            product.getReservedQuantity() + event.getQuantity());
        productRepo.save(product);

        // Save reservation record (needed for compensation later)
        Reservation reservation = new Reservation(orderId,
            event.getProductId(), event.getQuantity(), "RESERVED");
```

```
        reservationRepo.save(reservation);

        // Publish success event
        publishEvent("inventory-reserved",
            Map.of("orderId", orderId, "productId", event.getProductId()));

        log.info("Stock reserved for order: {}, product: {}, qty: {}",
                 orderId, event.getProductId(), event.getQuantity());
    }

    // ------------------------------------------------------
    // COMPENSATING STEP: Release stock when payment fails
    // This UNDOES the reservation made above
    // ------------------------------------------------------
    @KafkaListener(topics = "inventory-release", groupId = "inventory-service-group")
    @Transactional
    public void onInventoryRelease(String message) {
        InventoryReleaseEvent event = parseEvent(message, InventoryReleaseEvent.class);
        Long orderId = event.getOrderId();

        Reservation reservation = reservationRepo.findByOrderId(orderId)
            .orElse(null);

        if (reservation == null || "RELEASED".equals(reservation.getStatus())) {
            log.warn("Reservation not found or already released for order: {}", orderId);
            return;  // Idempotency: don't release twice
        }

        // UNDO the reservation
        Product product = productRepo.findByProductId(reservation.getProductId()).get();
        product.setAvailableQuantity(
            product.getAvailableQuantity() + reservation.getQuantity());
        product.setReservedQuantity(
            product.getReservedQuantity() - reservation.getQuantity());
        productRepo.save(product);

        // Mark reservation as released
        reservation.setStatus("RELEASED");
        reservationRepo.save(reservation);

        log.info("Stock released for order: {}", orderId);
    }

    private void publishEvent(String topic, Map<String, Object> payload) {
        try {
            kafkaTemplate.send(topic, objectMapper.writeValueAsString(payload));
        } catch (Exception e) {
            throw new RuntimeException("Failed to publish event to: " + topic, e);
        }
    }
}
```

Chapter 9

Payment Service — Complete Code

PAYMENT SERVICE LISTENS TO 1 TOPIC:

- 'payment-requested' — Charge the customer

```
@Service
@Slf4j
public class PaymentService {

    @Autowired private PaymentRepository paymentRepo;
    @Autowired private KafkaTemplate<String, String> kafkaTemplate;
    @Autowired private ObjectMapper objectMapper;

    // ------------------------------------------------------
    // FORWARD STEP: Process payment when inventory is reserved
    // ------------------------------------------------------
    @KafkaListener(topics = "payment-requested", groupId = "payment-service-group")
    @Transactional
    public void onPaymentRequested(String message) {
        PaymentRequestedEvent event = parseEvent(message, PaymentRequestedEvent.class);
        Long orderId = event.getOrderId();

        // IDEMPOTENCY CHECK: Never charge the same order twice!
        if (paymentRepo.existsByOrderId(orderId)) {
            log.warn("Payment already exists for order: {}. Skipping.", orderId);
            return;
        }

        // Create payment record FIRST with PENDING status
        // This is important: if server crashes after charge but before save,
        // we can detect it on retry
        Payment payment = new Payment(orderId, event.getAmount(), PaymentStatus.PENDING);
        payment = paymentRepo.save(payment);

        try {
            // Call external payment gateway (Stripe, PayPal, Razorpay, etc.)
            String gatewayRef = paymentGateway.charge(orderId, event.getAmount());

            // Update payment to COMPLETED
            payment.setStatus(PaymentStatus.COMPLETED);
            payment.setGatewayRef(gatewayRef);
            paymentRepo.save(payment);

            // Publish success
            publishEvent("payment-completed",
                Map.of("orderId", orderId, "gatewayRef", gatewayRef));

            log.info("Payment completed for order: {}, ref: {}", orderId, gatewayRef);

        } catch (PaymentGatewayException e) {
            // Payment gateway rejected the charge
            payment.setStatus(PaymentStatus.FAILED);
            paymentRepo.save(payment);
```

```
            publishEvent("payment-failed",
                Map.of("orderId", orderId, "reason", e.getMessage()));

            log.warn("Payment failed for order: {} reason: {}", orderId, e.getMessage());
        }
    }

    // ------------------------------------------------------
    // COMPENSATION (optional): Refund if needed
    // Note: In our flow, payment only succeeds if everything works,
    // so refund is mainly for edge cases or manual processes
    // ------------------------------------------------------
    @KafkaListener(topics = "payment-refund", groupId = "payment-service-group")
    @Transactional
    public void onPaymentRefund(String message) {
        PaymentRefundEvent event = parseEvent(message, PaymentRefundEvent.class);

        paymentRepo.findByOrderId(event.getOrderId()).ifPresent(payment -> {
            if (!"REFUNDED".equals(payment.getStatus().name())) {
                // Issue refund via payment gateway
                paymentGateway.refund(payment.getGatewayRef());
                payment.setStatus(PaymentStatus.REFUNDED);
                paymentRepo.save(payment);
                log.info("Refunded payment for order: {}", event.getOrderId());
            }
        });
    }
}
```

Chapter 10

Idempotency — Handling Duplicate Events

WHAT IS IDEMPOTENCY? (Simple explanation)

Idempotency means: doing the same operation 10 times gives the SAME result as doing it once.

Example: Pressing the elevator button 10 times doesn't summon 10 elevators. The elevator comes once. The extra button presses are ignored.

In Kafka: a message can be delivered MORE THAN ONCE (Kafka's 'at-least-once' guarantee). Your code must handle duplicate messages gracefully — process it once, ignore the rest.

WHY DOES KAFKA SEND DUPLICATE MESSAGES?

- Consumer crashes AFTER processing but BEFORE committing the offset to Kafka.
- Kafka thinks the message was not processed (offset not committed).
- Kafka redelivers the message to the next consumer instance.
- Result: message is processed TWICE.

```
// WITHOUT IDEMPOTENCY — DANGEROUS!
@KafkaListener(topics = "order-created")
public void onOrderCreated(String message) {
    OrderCreatedEvent event = parse(message);
    product.setQuantity(product.getQuantity() - event.getQuantity());
    // If this message comes twice, stock is reduced TWICE! Customer exploited us.
}

// WITH IDEMPOTENCY — SAFE!
@KafkaListener(topics = "order-created")
@Transactional
public void onOrderCreated(String message) {
    OrderCreatedEvent event = parse(message);

    // Check: did we already process this exact order?
    if (reservationRepo.existsByOrderId(event.getOrderId())) {
        log.warn("DUPLICATE MESSAGE. Already processed order {}. Skipping.",
                 event.getOrderId());
        return;  // Exit immediately. Do nothing.
    }

    // First time seeing this orderId — proceed normally
    product.setQuantity(product.getQuantity() - event.getQuantity());
    reservationRepo.save(new Reservation(event.getOrderId(), ...));
    // Now even if message comes 100 times, stock is only reduced once!
}
```

■ TIP: Always use a UNIQUE constraint on orderId in your reservation/payment tables. This gives you database-level idempotency as a safety net, even if your code check is missed.

Chapter 11

Summary & Quick Reference Card

WHEN TO USE SAGA PATTERN

USE Saga When	AVOID Saga When
Multiple microservices each with their own DB need to participate in one business operation	You have a single service with one database — just use @Transactional
You want eventual consistency (not instant) across service boundaries	You need strong immediate ACID consistency and can use one DB
Long business processes: order placement, hotel booking, loan approval	Simple CRUD — creating a user profile, updating a setting
You are already using event-driven architecture with Kafka/RabbitMQ	Team is unfamiliar with eventual consistency — high risk of bugs

THE 5 GOLDEN RULES OF SAGA PATTERN

Rule 1	**Each service has its OWN database** Never share a database between microservices. This is the foundation.
Rule 2	**Use the Outbox Pattern** Never write to DB and Kafka in separate operations. Use a local outbox table to prevent dual-write problems.
Rule 3	**Every step must be Idempotent** Your Kafka listener must handle duplicate messages without side effects. Check before processing.
Rule 4	**Design Compensation FIRST** Before coding the forward step, know exactly how to UNDO it. If you can't undo it, you can't use Saga.
Rule 5	**Track Saga State** Keep a saga_state table so you can always answer: where is this saga? Is it stuck? Can you retry?

QUICK PROBLEM-SOLUTION MAP

Problem / Question	Solution
Kafka is down when I publish an event	Use Outbox Pattern — save event to DB first, publish when Kafka recovers
Same Kafka message processed twice	Idempotency — check if orderId already exists before processing
Why can't services share one DB?	Independence, separate scaling, no coupling. One DB = distributed monolith, not microservices

Saga is stuck at PENDING	Add saga timeout: if saga stays in-progress > X minutes, auto-cancel and compensate
How do I debug a saga across services?	Distributed tracing (Zipkin/Jaeger) with correlation IDs. Every event carries the same orderId
Compensation failed — stock was released but order still shows PENDING	Retry compensation via Dead Letter Queue (DLQ) + saga_state table for manual review

You now know: Why Saga exists, why separate DBs, how HLD connects services, the complete Saga flow with compensation, how to solve Kafka late-read with Outbox Pattern, full LLD schemas and APIs, every line of Java code in all 3 services, and idempotency.

Start small: implement Order + Inventory first, get the happy path working, then add compensation and the Outbox Pattern step by step.